6.1 Skills概念与ClawHub

> **生成模型**：OpenAI GPT-5.3 Codex（openai/gpt-5.3-codex）
> **Token 消耗**：输入约 12k，输出约 6k（本节，估算值）

到第 6 章，我们终于来到 OpenClaw 最“好玩”的一层：Skills 生态。

前面几章你已经看到 OpenClaw 的主干能力了：会话、路由、工具调用、消息处理都很强，但这些更像一台装配好的“通用机器”。真正让它变成“你自己的机器”的，是 Skills。

如果用一句话讲清楚：**Skill 是给 Agent 提供“领域操作说明书”的热插拔能力模块。**

这里有三个关键词：

- **热插拔**：不改主程序，不重写核心代码，按需装、按需开。
- **能力模块**：一个 Skill 只解决一类问题，比如 GitHub 流程、天气查询、浏览器自动化。
- **说明书**：Skill 主要是写给模型读的，不是写给编译器读的。

这一节我们不进入底层源码细节（那是第 29 章重点），而是站在“使用者+系统设计直觉”的视角，搞懂下面几个问题：

1. Skill 到底是什么、和 Tool 有什么关系；
2. OpenClaw 里 Skill 有哪些类型，冲突时谁覆盖谁；
3. ClawHub 这个技能市场怎么用；
4. 平时如何安装、启用、禁用、排查 Skill 状态；
5. 怎么在 CLI 命令和聊天指令里高效调用 Skills。

6.1.1 Skill 到底是什么：不是“插件代码”，而是“可执行知识包”

很多同学第一次接触 OpenClaw 会有个误解：Skill 是不是类似 VS Code 扩展，要写一堆 JS/TS 代码？

实际并不一定。

在 OpenClaw 里，一个 Skill 最小形态可以只是一个目录，里面放一个 `SKILL.md` 。这个文件告诉模型：

- 你解决什么问题；
- 你依赖什么外部条件（命令行工具、环境变量、系统平台）；
- 面对具体任务时，推荐怎么操作；
- 有哪些坑要避开。

你可以把它理解成“给 Agent 的标准作业流程（SOP）文档”。

Skill 和 Tool 的分工

这是非常关键的概念：

- **Tool（工具）**：提供“手脚”，比如 `bash` 、文件读写、浏览器控制；
- **Skill（技能）**：提供“方法论”，告诉 Agent 什么时候用哪只手、先后顺序怎么排。

举个很直观的例子：

- 没有天气 Skill，模型可能“凭常识瞎猜”天气；
- 有天气 Skill 后，模型会按说明走 `curl` 去查询真实服务（如 wttr.in）。

所以 Skill 解决的是“会不会做、会不会稳做”的问题，不是“能不能执行系统调用”的问题。

> **衍生解释：为什么这是教科书外但很重要的概念？**
>
> 在传统 CS 课程里，我们更常见的是“函数调用接口”或“插件 API”。Skill 这种“自然语言驱动的能力注入”属于 LLM 时代的新型扩展机制：不是让程序调用程序，而是让模型通过上下文学会一套行为策略，再去调用工具执行。这是一种“文档即能力”的工程模式。

6.1.2 Skills 的类型： Extra / Bundled / Managed / Workspace

根据来源和作用域，OpenClaw 里的 Skill 分为四类，按优先级从低到高排列：

1）Extra Skills（额外技能）

- 通过配置指定的额外目录提供；
- 优先级最低，通常由插件或自定义集成注入；

- 适合补充一些非标准来源的技能。

实际使用中很少直接接触这一层，但它的存在让系统更灵活。

2）Bundled Skills（内置技能）

- 随 OpenClaw 一起分发，当前版本内置 52 个技能；
- 开箱就有，覆盖开发工具、通信平台、笔记/知识、智能家居、AI/模型、媒体、工具、生态等 8 大类；
- 例如： `coding-agent` 、 `github` 、 `weather` 、 `discord` 、 `obsidian` 、 `clawhub` 等。

你可以把它当成“官方预装包”。

3）Managed Skills（托管技能）

- 通常放在 `~/.openclaw/skills` ；
- 常由 ClawHub 安装和管理；
- 跨项目复用最方便。

如果你在多个仓库都需要同一种能力，比如 GitHub 自动化或某个团队内部发布流程，这类最合适。

4）Workspace Skills（工作区技能）

- 放在当前项目的 `<workspace>/skills` ；
- 只在这个项目生效；
- 优先级最高，特别适合团队私有规范。

比如 A 团队要求“发版前必须先跑某三项检查”，B 团队完全不同，那就把规则写成 workspace skill，互不干扰。

优先级：谁覆盖谁

当存在同名 Skill 时，OpenClaw 会按优先级选择：

```
workspace > managed > bundled > extra
```

这条规则很实用：

- 你可以先用内置 Skill 起步；
- 觉得不够贴合，再用 managed 版本增强；
- 项目内再用 workspace 版本做最终定制。

这本质上是一种“逐层覆盖”的配置策略，与第 29 章源码分析中的 `loadSkillEntries` 四来源合并逻辑完全一致。

6.1.3 ClawHub 是什么：Skills 的“应用市场 + 分发中心”

如果把 Skills 生态比作手机 App 生态，ClawHub 就是应用商店。

它至少做三件事：

1. **发现（Discover）**：让你搜索可用技能；
2. **安装（Install）**：把技能装到本地可识别目录；
3. **版本与分发（Distribute）**：方便作者发布、用户更新。

为什么 ClawHub 重要

如果没有 ClawHub，Skill 分享会变成“发压缩包+手工拷目录”，体验和可维护性都很差。ClawHub 把这件事产品化了：

- 统一入口；
- 统一安装命令；
- 统一命名和可发现性。

对于学习者来说，最大的价值是：**你可以先站在成熟 Skill 的肩膀上，理解一个好 Skill 长什么样，再自己写。**

6.1.4 安装与发现：CLI 视角的一套标准动作

下面给你一套很实用的“固定动作”，建议直接记下来。

第一步：看本地技能现状

```
openclaw skills list
openclaw skills list --eligible
openclaw skills info github
openclaw skills check
```

- `list`：看有哪些技能；
- `list --eligible`：看当前环境真正可用的技能；
- `info <name>`：看单个技能描述、依赖；
- `check`：做整体健康检查。

这里的 `eligible` 很关键。它告诉你“理论上有”不等于“实际上可用”。

例如 `github` Skill 常依赖 `gh`，如果你没装 `gh`，那技能就可能处于不可用状态。

第二步：在 ClawHub 搜索

```
clawhub search "github"
clawhub search "weather"
clawhub list
```

第三步：安装技能

```
# 先安装 ClawHub CLI（如尚未安装）
npm i -g clawhub

# 安装常见 Skills
clawhub install coding-agent
clawhub install github
clawhub install weather
clawhub install discord
clawhub install canvas
```

第四步：回到 OpenClaw 验证

```
openclaw skills list --eligible
openclaw skills info coding-agent
openclaw skills info weather
```

如果这里显示正常，基本就可以在对话里直接用了。

6.1.5 在聊天里怎么触发 Skill

很多人把 Skill 想成“要先手动打开插件开关”。实际上，常见流程是：

- 你自然语言提任务；
- Agent 根据任务类型和已加载 Skills 决定采用哪套策略；
- 再配合工具完成执行。

比如你可以直接说：

```
帮我看一下这个仓库最近 10 个 PR，按风险从高到低排序，并指出谁还没过 CI。
```

如果 `github` Skill 可用，模型会倾向于按照 Skill 推荐流程去用 `gh` 完成，而不是只给你“纸面建议”。

再比如：

```
查一下上海未来三天的天气，按通勤建议给我一个穿衣清单。
```

有 `weather` Skill 时，模型会优先查真实天气服务，再输出建议。

> **衍生解释：命令式调用 vs 意图式调用**
>
> 传统系统常要求用户明确调用某个命令（命令式）。Skill 生态更偏“意图式”：你表达目标，系统根据上下文和能力快照选择策略。这个思想在智能体系统里非常重要，它降低了使用门槛，但前提是 Skill 设计要清晰可靠。

6.1.6 在配置里管理 Skill 开关与参数

当你不想“全自动”，或者需要对某些 Skill 注入配置，可以在 `openclaw.json` 的 `skills.entries` 做管理。

示例：

```
{
  skills: {
    entries: {
      github: {
        enabled: true
      },
      weather: {
        enabled: true
      },
      "coding-agent": {
        enabled: true
      },
      "my-company-skill": {
        enabled: true,
        apiKey: "$MY_COMPANY_API_KEY",
        env: {
          MY_COMPANY_API_KEY: "$MY_COMPANY_API_KEY"
        },
        config: {
          endpoint: "https://api.example.com",
          team: "infra"
        }
      }
    }
  }
}
```

这个配置里有几个常用点：

- `enabled`：快速开关；
- `env`：注入环境变量；
- `apiKey`：给技能透传密钥（通常配合环境变量）；
- `config`：放业务参数（地址、团队、阈值等）。

推荐习惯

- 密钥不要硬编码，统一放环境变量；
- 先在 workspace 里验证，再上 managed 全局；
- 给团队 skill 约定清晰的 `config` 字段名，避免口口相传。

6.1.7 Skill 状态管理：能装不等于能用

实战里最容易踩的坑不是“不会安装”，而是“装了但没生效”。

你可以用一个三层检查法：

第一层：发现层

- `openclaw skills list` 能看到吗？
- 名称是否和你调用时一致？

第二层：资格层

- `openclaw skills list --eligible` 里有吗？
- 依赖命令（如 `gh` 、 `curl` ）是否存在？

第三层：行为层

- 你提出对应任务时，Agent 的执行路径是否体现了该 Skill 的策略？
- 输出是否引用了真实外部信息，而非凭空生成？

常见状态问题和处理

1. **Skill 显示存在，但不 eligible**
   - 多数是外部依赖没装。
   - 处理：按 `skills info` 提示补齐命令行依赖。
2. **同名 Skill 行为和预期不一致**
   - 可能是被高优先级来源覆盖了。
   - 处理：检查 workspace/managed 是否有同名版本。
3. **启用了 Skill，效果仍像没启用**
   - 可能是任务描述过于模糊，没触发该技能场景。
   - 处理：任务描述里加上明确目标和约束。

例如把“帮我看看 GitHub”改成“帮我用 gh 检查最近 5 个 PR 的 CI 状态并列出失败原因”。

6.1.8 一个完整示例：从 0 到可用

这里我们做一遍完整流程，目标是启用 `github` Skill。

1）安装依赖与技能

```
# 安装 GitHub CLI
brew install gh

# 登录 GitHub
gh auth login
gh auth status

# 安装 skill
clawhub install github
```

2）检查可用性

```
openclaw skills info github
openclaw skills list --eligible
```

3）在对话中使用

```
请检查仓库 owner/repo 最近 5 个 PR，输出：PR 编号、作者、CI 状态、是否可合并。
```

如果一切正常，Agent 会倾向调用与 GitHub 流程相关的操作路径，而不是只“泛聊 GitHub 概念”。

6.1.9 给学习者的实践建议：先“会用”，再“会造”

你现在最应该做的不是立刻写复杂自定义 Skill，而是先把这三件事练熟：

1. **会判断 Skill 是否真的生效**（`eligible` + 行为观察）；
2. **会做最小配置管理**（`enabled`、`env`、`config`）；
3. **会选择作用域**（workspace 还是 managed）。

等你这三步稳定了，再进入 6.3 自定义 Skill，会非常顺。

因为写 Skill 最大难点其实不是语法，而是“把模糊经验变成稳定可复用的操作说明”。

本节小结

这一节你可以带走 5 个核心结论：

- Skill 是 OpenClaw 的能力扩展层，本质是面向 LLM 的“可执行知识包”；
- Tool 负责执行能力，Skill 负责行为策略，两者是互补关系；
- Skill 来源有四种：extra / bundled / managed / workspace，优先级是 `workspace > managed > bundled > extra` ；
- ClawHub 是 Skills 的发现与分发中心，建议形成“搜索→安装→验证”的固定流程；
- 技能管理的关键不在“装没装”，而在“是否 eligible、是否按预期行为执行”。

下一节我们就进入你最关心的部分：把 `coding-agent` 、 `github` 、 `weather` 、浏览器自动化、 `discord` 、 `canvas` 这些常用 Skills 一次讲透，直接上手实战。